



**Faculty of Engineering & Technology Electrical & Computer Engineering
Department**

Computer vision

ENCS5343

Course project's report

Prepared by:

Leen Ahmad, 1212028, section 1.

Mai zaid, 1210260, section 2.

Instructor: Aziz Qaroush.

25th January 2025.

Abstract:

This project explores the application of deep learning techniques for handwritten user identification using the AHAWP dataset. The tasks focused on designing and training a custom CNN, applying data augmentation, testing a pre-trained ResNet-50 model, and leveraging transfer learning for improved performance. Through iterative optimization, the models demonstrated progressive improvement in accuracy. Data preprocessing and augmentation strategies played a significant role in enhancing generalization, while fine-tuning pre-trained architectures provided robust solutions for handling complex data.

Table of Contents

Abstract:	- 2 -
Table of figures:	- 4 -
Introduction:	- 5 -
Dataset Description	- 5 -
Convolutional Neural Networks (CNN)	- 5 -
Methods	- 5 -
Technologies used:	- 6 -
Experimental setup:	- 7 -
Methodology:	- 7 -
Tasks:	- 7 -
Task 1:	- 7 -
Task 2:	- 11 -
Task 3:	- 14 -
Task 4:	- 15 -
Results analysis:	- 17 -
Conclusion:	- 18 -

Table of figures:

Figure 1: First CNN results. _____	- 9 -
Figure 2: preprocessing result. _____	- 10 -
Figure 3: preprocessing result. _____	- 10 -
Figure 4: task 1 result. _____	- 11 -
Figure 5: effect of zoom ranges on both test accuracy and loss. _____	- 12 -
Figure 6: effect of height ranges on both test accuracy and loss. _____	- 12 -
Figure 7: effect of width ranges on both test accuracy and loss. _____	- 12 -
Figure 8: effect of rotation range on testing accuracy and loss. _____	- 13 -
Figure 9: effect of shear range on testing accuracy and loss. _____	- 13 -
Figure 10: Data augmentation result. _____	- 14 -
Figure 11: task 3 result. _____	- 15 -
Figure 12: task 4 result. _____	- 16 -

Introduction:

The objective of this project is to utilize deep learning techniques to create a model that predicts different anonymous users from their handwriting. The project involves four key tasks: First, designing, training, and tuning custom CNN architectures to identify the simplest yet most effective network, accompanied by visualizations of accuracy and loss. Second, improving the selected custom CNN model by applying data augmentation techniques to enhance generalization and comparing the performance with the initial results. Third, training a well-known CNN architecture with data augmentation and evaluating its performance against the custom CNN. Finally, leveraging transfer learning by fine-tuning a pre-trained CNN on the given dataset, followed by a performance comparison with the results of the previous tasks. Each step focuses on improving model accuracy and robustness while analyzing the effect of various techniques.

Dataset Description

The AHAWP dataset (Arabic Handwritten Automatic Word Processing) is a benchmark dataset designed to aid the development of handwritten Arabic text recognition systems. It includes 10 words written by 82 individuals, with each contributing 10 samples per word, totaling 8,144 images.

Convolutional Neural Networks (CNN)

Convolutional Neural Networks (CNN) are a class of deep learning models specifically designed for processing data with grid-like topology, such as images. CNNs are particularly effective for image recognition and classification tasks due to their ability to automatically and adaptively learn spatial hierarchies of features from input images. A typical CNN architecture comprises multiple layers, including convolutional layers, pooling layers, and fully connected layers. The convolutional layers apply a set of filters to the input image to extract features such as edges, textures, and patterns. These features are then down sampled by the pooling layers to reduce their dimensionality while preserving essential information. Finally, the fully connected layers map the extracted features to the output classes or predictions.

The power of CNNs lies in their capacity to capture complex patterns and dependencies in visual data, making them ideal for tasks like handwritten text recognition. By leveraging the hierarchical structure of CNNs, we can build robust models that generalize well to new, unseen data, which is crucial for achieving high accuracy in real-world applications.

Methods

To enhance the performance of our CNN network, we employed several key techniques:

- **Batch Normalization:** This technique was applied to normalize the inputs of each layer to stabilize and accelerate the training process.
- **Epochs:** We trained the network over multiple epochs, refining the model parameters with each pass through the dataset to achieve optimal accuracy.

- **Learning Rate:** An appropriate learning rate was crucial for effective training, controlling the step size of parameter updates to balance between convergence speed and stability.
- **Data Augmentation:** Preprocessing techniques, such as rotation, scaling, and flipping of images, were used to artificially expand the dataset and improve generalization.
- **Transfer Learning:** By leveraging pre-trained models, we were able to transfer learned features from large datasets to our specific task, thereby enhancing performance with limited data.

In our data augmentation and transfer learning processes, we utilized the ResNet50 architecture and the ImageNet dataset. ResNet50, with its 50 layers, is known for its residual learning framework, which helps in training deeper networks by addressing the vanishing gradient problem. Leveraging the pre-trained ResNet50 model on the extensive ImageNet dataset allowed us to benefit from the learned features and patterns that are crucial for visual recognition tasks. This approach not only improved our model's performance but also reduced the computational cost and time required to train the network from scratch.

Technologies used:

- **Programming Language:** Python.
- **Deep Learning Frameworks:** TensorFlow (Keras) for the first and second task and PyTorch for the third and fourth.
- **Data Preprocessing Libraries:** Pillow (PIL), Scikit-image.
- **Machine Learning Libraries:** Scikit-learn.
- **Visualization Tools:** Matplotlib.
- **Model Techniques:** CNN, Data Augmentation, Transfer Learning (ImageNet).
- **Optimization and Regularization:** Adam Optimizer, Dropout, Label Smoothing.
- **General Utilities:** NumPy, OS Module.
- **Platform and Hardware:** All experiments were performed on a system with GPU acceleration (CUDA) using Kaggle platform.

Experimental setup:

Methodology:

Dataset

The dataset used for this project is the *AHAWP* handwriting dataset, consisting of isolated word images from multiple users. Key characteristics of the dataset include:

- **Number of Classes:** 82 distinct user classes.
- **Input Size:** Images were resized to 256×256 pixels for preprocessing for task 3 and 4 and 32×32 for task 1 and 2.
- **Preprocessing:** Images were converted to grayscale, binarized using Otsu's thresholding, inverted to emphasize handwriting, mirrored, and resized to meet the input requirements of the CNN models.

Evaluation Metrics

The following metrics were employed to evaluate the performance of the models:

1. **Accuracy:** The percentage of correctly classified samples, used as the primary metric for evaluating the model's overall performance.
2. **Loss:** The categorical cross-entropy loss was used to measure the divergence between predicted probabilities and true labels, providing insights into the model's optimization progress.
3. **Generalization:** Validation accuracy and loss curves were monitored to assess overfitting or underfitting.

Tasks:

Task 1:

Build and train a custom CNN network.

The process of creating an agreeable CNN involved many trials and error to find the best approach. The main challenges we faced, was the fact that the data we have is only 8144 images, split to three subcategories -training 60%, validation 20% testing 20%", this data is generally considered limited or too small when dealing with a complex design such as the CNN.

The following is a simple walk through of the fix and edit we did on the architecture until we reached a good kick-off to continue with for the data augmentation.

The architecture we initially began with was as follows:

Category	Parameter	Value/Type
Architecture	Number of layers	11 layers (3 Conv + 3 Pooling + 3 Dropout + Flatten + Output)
	Types of layers	Convolutional, Pooling, Batch Normalization, Flatten, Fully Connected
	Activation functions	ReLU (softmax for the output layer)
Convolutional Layer Parameters	Number of filters	64, 128, 256
	Filter size	(3, 3)
	Stride	1
	Padding	Zero padding (same padding).
Pooling Layer Parameters	Pool size	(2, 2)
	Pool type	Max Pooling
Fully Connected Layer Parameters	Number of neurons	4096, 1024, 512
Training Hyperparameters	Learning rate	0.0001
	Batch size	16
	Epochs	20
	Optimizer	Adam (Loss function: categorical cross entropy loss)
	Overfitting Handling	0.6 (after each fully connected layer)

The results obtained from this initial setup were acceptable the testing accuracy was 28.2 and validation accuracy was 27.2 which means they're close indicating no overfitting happened. However, the results were still not ultimate.

Figure 1 shows the graphs obtained for the training and validation loss vs epochs, the second graph is for the training and testing accuracy vs epochs.



Figure 1: First CNN results.

Another good notice obtained from the graphs, that both losses were steadily decreasing, which means they are approaching convergence.

The following outlining steps summarize the process of improving the architecture:

- Simplifying the architecture:** at first, we thought the architecture was more complex than the data set required, so we did two experiments the first was to decrease the number of filters introduced in the convolutional layers (**from 64, 138, 256 to 32, 64, 128**) this experiment helped in nothing the accuracy was around 24% which is slightly less than the original. The second experiment was to simplify fully connected layer architecture, the idea was primarily generated due to our thoughts that the dataset we had was simpler than the CNN we built, so we also did this step in two stages: the first was to reduce the number of convolutional layer to two layers of 512 and 256 neuron, however it negatively reflected the accuracy and loss “decreased the accuracy and increased the loss rate”, the second step was to reduce the number of neurons (**from 4096, 1024, 512 to 512, 256, 128 or 1024, 512, 256**) both reductions sharply decreased the accuracy to reach 14 for the first and 8 for the latter. Thus, both experiments failed to enhance the results we had so both were eliminated.
- Batch size:** the impact of batch normalization relays in stabilizing the input sent to the neurons by controlling the covariant internal shift which ultimately helps in accelerating the training process and reaching better convergence. However, significantly increasing the batch size could lead to failure in generalizing and being computationally expensive. Thus, the tradeoff between increasing it or leaving it as it is was confusing, so we increased it to 32 only. This increment helped in stabilizing the output, giving better accuracy so we chose to modify the originally attempted one to 32.
- Number of epochs:** the urge to increase this feature, was due to our thoughts that the model is not learning effectively because of the small number of epochs. At first, we increased it to 32, which gave better results, where accuracy reached 36, which was a good sign of increasing it. However, the over increment in the number of epochs would

lead to the failure of model to generalize and making it memorize the patterns more thus we kept it as it is.

- **Learning rate:** the experiment on this feature was short, we simply increased it from 0.0001 to 0.001 because we thought that the primary value was too small. The result was not as we wished; it decreased the accuracy to 16 which made us eliminate it. Another approach we tried is introducing the adaptive learning rating using ReduceLROnPlateau. However, the results were worse, it decreased the accuracy to 8%, thus we kept the learning rate as it is.
- **Dropout:** we tried to increase the dropout rate to 0.7 try and prevent the reliance of the network on specific neurons and to make it learn more robust features. However, based on our dataset the increment in this feature was not suitable for it, the accuracy decreased sharply. We tried to decrease it to 0.5, the results were not better, and overfitting was highly noticed.

After these experiments, we thought about introducing some preprocessing on the images before introducing them to the network, the preprocessing included converting the image to grayscale “making its dense 1 channel”, inverting, reflecting. Resizing and binarization. This step really helped in improving the accuracy, it reached 40%. .

The following two figures show the result of the introduction of the preprocessing step by step on two random words for two different users.



Figure 2: preprocessing result.



Figure 3: preprocessing result.

Finally, we tried increasing the number of epochs to 50 with preprocessing. This step gave us the best results the accuracy reached 50.49% so we stopped our experiments, and the following plots were conducted.

To summarize, the key changes conducted on the dataset were to increase the batch size to 32, increase the number of epochs to 50, introducing preprocessing and using 32x32 images.

Figure 4 shows the results of this simulation:

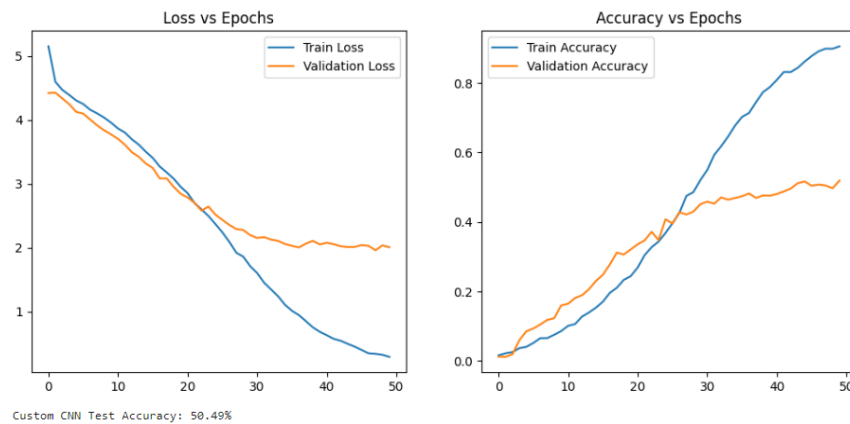


Figure 4: task 1 result.

The figure shows a significant boost in training and validation accuracy with better convergence. This improvement highlights effective adjustments to the model architecture, preprocessing, or training strategy.

Both training and validation losses show a smoother decline, indicating more stable training. The consistent gap between the two is expected in a custom CNN mode, and will be better handled with fine tuning later on.

Task 2:

Retrain the network selected from Task 1 after doing data augmentation.

To choose the best data augmentation, we divided this task to four subcategories as follow:

- **Zoom effect:** To determine the optimal zoom level for image augmentation, we tested a range of values: [0.1, 0.2, 0.3, 0.4, 0.5]. We evaluated the impact of each zoom level on the model's accuracy and loss rate. As depicted in Graph 5, a zoom value of 0.1 yielded the highest accuracy and the lowest loss rate, making it the most effective choice for augmentation. Consequently, a zoom factor of 0.1 was selected as the optimal value for this parameter.

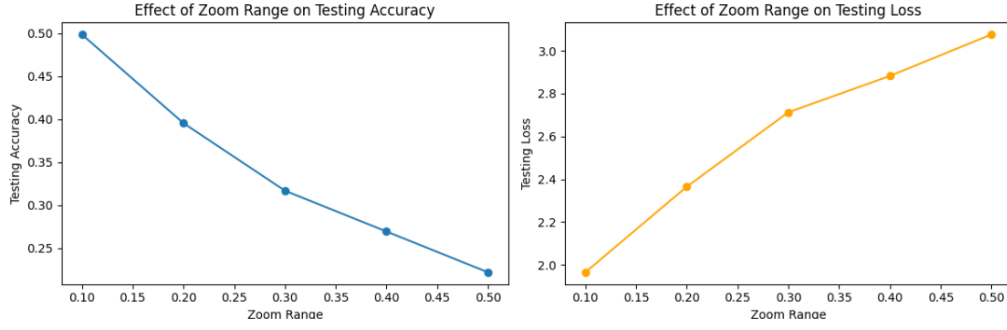


Figure 5: effect of zoom ranges on both test accuracy and loss.

- Height effect:** To evaluate the impact of height variations, we fixed the zoom level at 0.1, as determined in the previous section, and tested various height shift values to identify the optimal setting for model performance. As illustrated in Figure 6, a height shift value of 0.1 resulted in the best performance, achieving the highest accuracy and optimal model capacity. Consequently, the height shift value was fixed at 0.1 for augmentation.

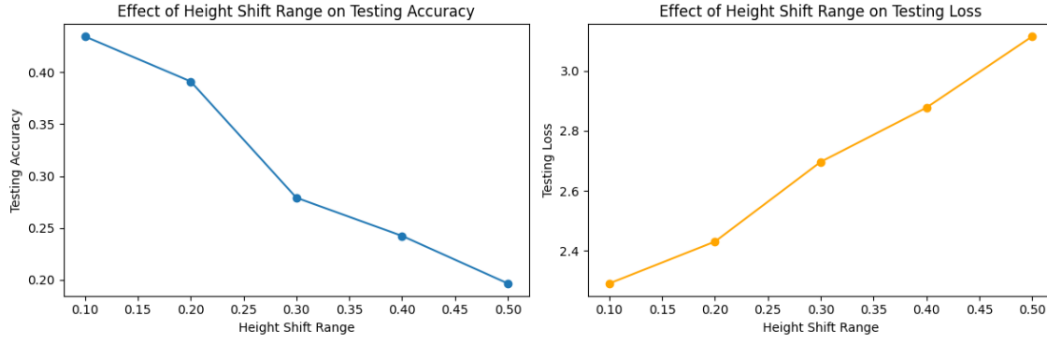


Figure 6: effect of height ranges on both test accuracy and loss.

- Width effect:** To evaluate the impact of width variations, we fixed the zoom level at 0.1 and height at 0.1, as determined in the previous sections, and tested various width shift values to identify the optimal setting for model performance. As illustrated in Figure 7, a width shift value of 0.1 resulted in the best performance, achieving the highest accuracy and optimal model capacity. Consequently, the width shift value was fixed at 0.1 for augmentation.

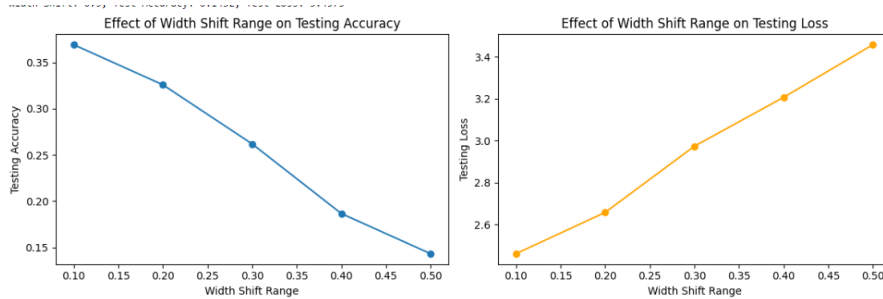


Figure 7: effect of width ranges on both test accuracy and loss.

- **Rotation effect:** To evaluate the impact of rotation variations, we fixed the zoom level at 0.1, height at 0.1 and the width at 0.1, as determined in the previous sections, and tested various rotation shift values to identify the optimal setting for model performance. As illustrated in Figure 8, a rotation shift value of 10 resulted in the best performance, achieving the highest accuracy and optimal model capacity. Consequently, the rotation shift value was fixed at 10 for augmentation.

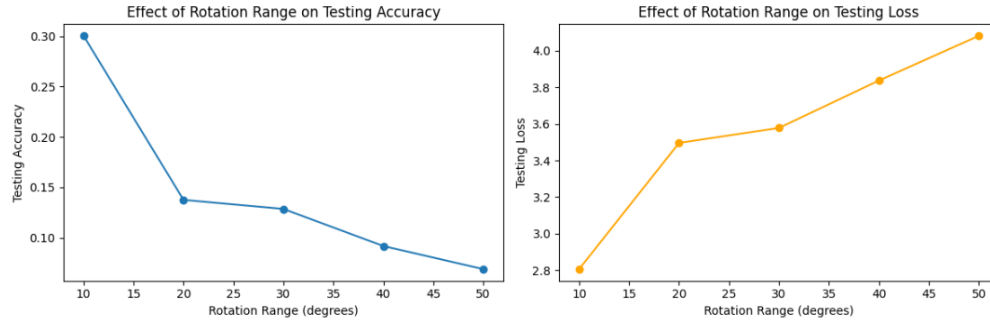


Figure 8: effect of rotation range on testing accuracy and loss.

- **Shearing effect:** To evaluate the impact of shear variations, we fixed the zoom level at 0.1, height at 0.1, the width at 0.1 and rotation at 0.1, as determined in the previous sections, and tested various shear shift values to identify the optimal setting for model performance. As illustrated in Figure 9, a shear shift value of 0.3 resulted in the best performance, achieving the highest accuracy and optimal model capacity. Consequently, the rotation shift value was fixed at 0.3 for augmentation.

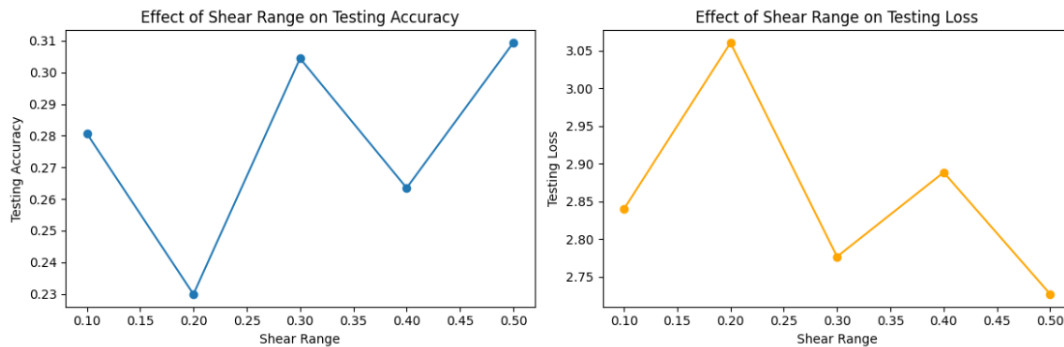


Figure 9: effect of shear range on testing accuracy and loss.

Thus, the final results that were used for the data augmentation were as follow:

Spatial transformation name	Value
Zoom range	0.1
Height shift	0.1
Width shift	0.1
rotation	10
shear	0.3

After analyzing the best possible candidates to give the best values for the data augmentation, the following plot “figure 10” shows the result of using data augmentation to enhance the model’s performance.

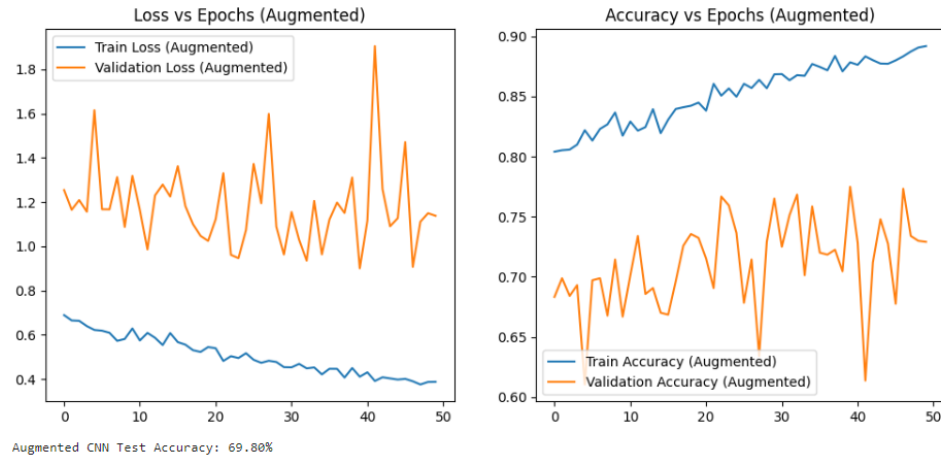


Figure 10: Data augmentation result.

The result shows that the testing accuracy increased significantly almost 20% more compared to task 1, which shows that the model is learning better and generalizing more.

However, regarding the losses, the training loss is steadily decreasing, showing effective learning. While for the validation loss, it fluctuates significantly after an initial decline, indicating overfitting or instability in learning.

Task 3:

In this section, the CNN selected was the ResNet50 architecture. This model was chosen after comparing it with other architectures and assessing their performance. ResNet30 was initially tested and provided moderate results, but it lacked the robustness required for the task. Similarly, AlexNet, while simpler and less computationally intensive, produced suboptimal results due to its limited depth and capacity. ResNet50, with its deeper architecture and residual connections, offered a better balance between complexity and performance, making it a suitable choice for this task. Its ability to effectively handle vanishing gradients and extract meaningful features contributed to its superior results compared to the other architectures tested.

Figure 11 shows the results obtained from this task.

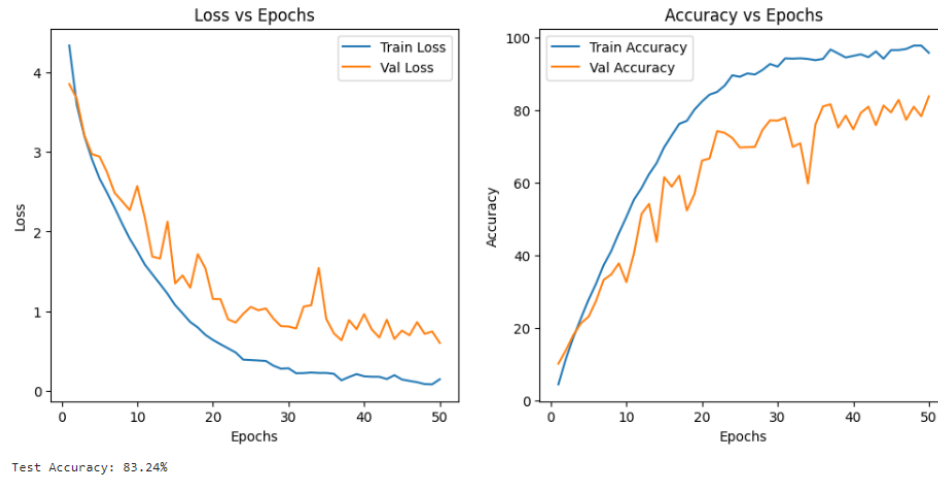


Figure 11: task 3 result.

As shown from the graph, the model showed better performance, increased the accuracy and the overall decrement and handling of the loss. The testing accuracy obtained from this part was 82.24%.

Task 4:

We used the pre-trained ResNet-50 model on ImageNet for our task. The initial layers (up to layer1) were frozen, while deeper layers (layer2, layer3, layer4, and the fully connected layer) were fine-tuned for our dataset. We customized the fully connected layer by adding a 512-unit dense layer with ReLU activation, a 50% dropout layer, and a softmax output layer for classifying 82 classes. Adam optimizer and label smoothing were used to improve generalization. Data augmentation techniques like rotation, shifting, shearing, and zooming enhanced training data diversity. This transfer learning approach combines “Feature extraction” and “Fine-tuning” to optimize performance.

Figure 12 shows the results obtained after running the code.

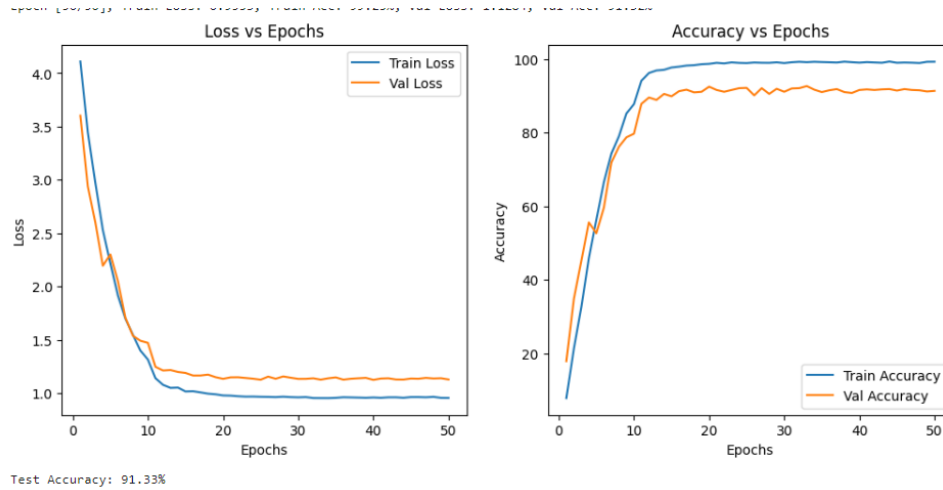


Figure 12: task 4 result.

As shown from the graph this task outperformed all other tasks, in handling the generalization and stabilization of the model. The test accuracy was 91.33%.

Results analysis:

1. Comparing task 1 and task 2:

The augmented CNN significantly outperformed the custom CNN, achieving a test accuracy of **69.80%** compared to **50.49%**. This improvement is due to the enhanced generalization from data augmentation. However, the augmented model showed instability in validation metrics, possibly due to aggressive augmentation or insufficient regularization. On the other hand, the custom CNN exhibited more stable and smoother training trends but did not generalize as well, underscoring the importance of augmentation for better performance.

2. Comparing task 2 and task 3:

The results from Task 3 (Test Accuracy: **83.24%**) demonstrate a consistent decline in both training and validation loss, alongside a smoother improvement in accuracy, culminating in a stable high validation accuracy plateau. This reflects effective learning with minimal overfitting. In contrast, the results from Task 2 (Augmented CNN Test Accuracy: 69.80%) reveal significant fluctuations in validation loss and accuracy, even as training metrics improve steadily. These variations indicate potential overfitting or instability in the augmented dataset, where the model faces challenges in generalizing to diverse augmented samples. The disparity in test accuracy underscores the difficulty of managing augmented data effectively without proper fine-tuning.

3. Comparing task 3 and task 4:

The model from task 4 achieves a significantly higher test accuracy of **91.33%**, compared to the task 3 model's **83.24%**. The loss and accuracy plots of the fourth show smoother and more consistent convergence, indicating better generalization and stability. In contrast, the third model exhibits higher fluctuations in validation loss and accuracy, reflecting potential overfitting or instability in training. The model from the fourth task demonstrates superior performance and robustness, making it the better choice for the given dataset.

Throughout the progression of tasks, the models demonstrated steady and significant improvement in performance. Starting with a baseline accuracy of around **50%**, each subsequent task achieved better results by incorporating refined architectures and improved data augmentation techniques. The second model reached approximately **70%** accuracy, showing enhanced generalization, while the third task achieved **83%**, reflecting stronger learning capabilities and better control over loss. Finally, the fourth model excelled with an impressive **91%** accuracy, demonstrating the highest level of generalization and stability. This consistent improvement highlights the effectiveness of iterative optimization, both in model design and training strategies, to achieve superior performance.

Conclusion:

This project highlighted the importance of iterative optimization in achieving improved performance across different tasks. The initial custom CNN achieved baseline accuracy, which was significantly enhanced through data augmentation. Transitioning to ResNet-50 architecture and employing transfer learning with fine-tuning demonstrated superior performance and stability. The final model outperformed all others, achieving 91.33% test accuracy and showcasing excellent generalization.

For future work, exploring advanced augmentation techniques, integrating generative adversarial networks (GANs) for dataset expansion, and experimenting with other state-of-the-art architectures like Vision Transformers could further improve performance. Additionally, deploying the model in real-world applications would validate its effectiveness in practical scenarios and uncover areas for refinement.